WEEK 3 - ASSIGNMENT

Objective:

Use Subqueries, CTEs, and Window Functions to analyze sales data from the Superstore dataset.

Tasks

Step 1: Setup Data

1. Import the Superstore dataset into a table named **superstore_raw**.

Step-1 : Download the superstore-dataset.csv from given resource.
Step-2 : Created a new database celebal_week3 in mysql workbench.
Step-3: Selected the superstore csv.
Step-4: Imported the dataset into a table named **superstore_raw**.
Step-5: check it by using SELECT

```
7
8  SELECT *
9  FROM superstore_raw
10 LIMIT 2;
11
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country
1	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	United States
2	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	United States

2. Create these 3 tables from it:
    a. customers

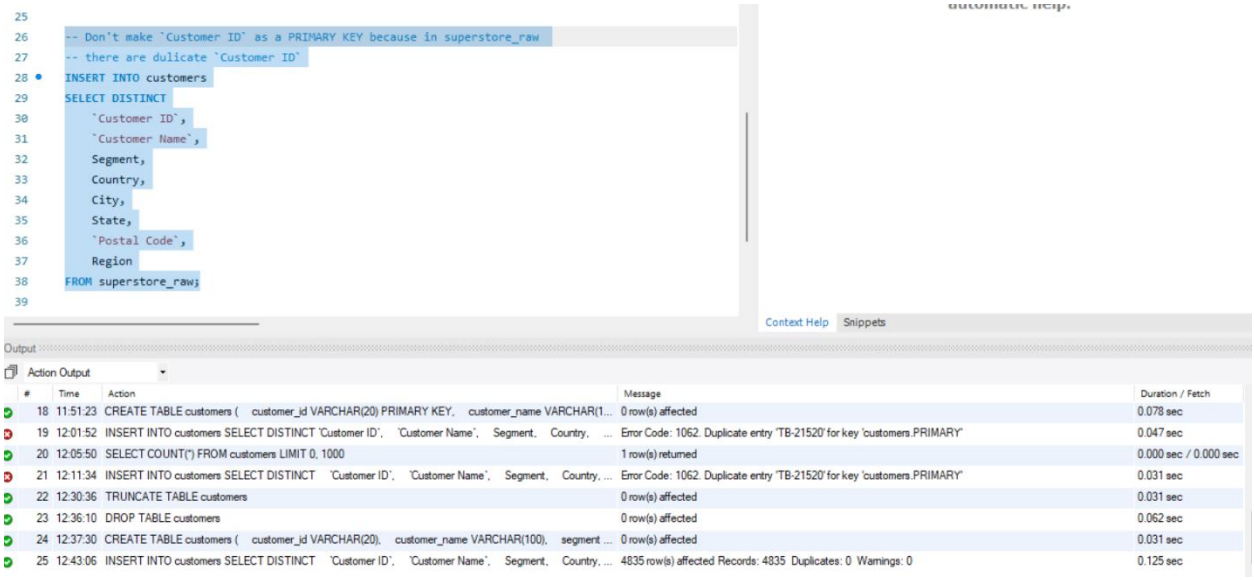
```
14
15 CREATE TABLE customers (
16     customer_id VARCHAR(20),
17     customer_name VARCHAR(100),
18     segment VARCHAR(50),
19     country VARCHAR(50),
20     city VARCHAR(50),
21     state VARCHAR(50),
22     postal_code INT,
23     region VARCHAR(20)
24 );
25
26
```

Context Help | Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
17	11:48:14	SELECT MAX(length(customer_id)) FROM superstore_raw LIMIT 0, 1000	Error Code: 1054. Unknown column 'customer_id' in 'field list'	0.000 sec
18	11:51:23	CREATE TABLE customers (customer_id VARCHAR(20) PRIMARY KEY, customer_name VARCHAR(1...	0 row(s) affected	0.078 sec
19	12:01:52	INSERT INTO customers SELECT DISTINCT 'Customer ID', 'Customer Name', Segment, Country, ...	Error Code: 1062. Duplicate entry 'TB-21520' for key 'customers.PRIMARY'	0.047 sec
20	12:05:50	SELECT COUNT(*) FROM customers LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
21	12:11:34	INSERT INTO customers SELECT DISTINCT 'Customer ID', 'Customer Name', Segment, Country, ...	Error Code: 1062. Duplicate entry 'TB-21520' for key 'customers.PRIMARY'	0.031 sec
22	12:30:36	TRUNCATE TABLE customers	0 row(s) affected	0.031 sec
23	12:36:10	DROP TABLE customers	0 row(s) affected	0.062 sec
24	12:37:30	CREATE TABLE customers (customer_id VARCHAR(20), customer_name VARCHAR(100), segment ...	0 row(s) affected	0.031 sec



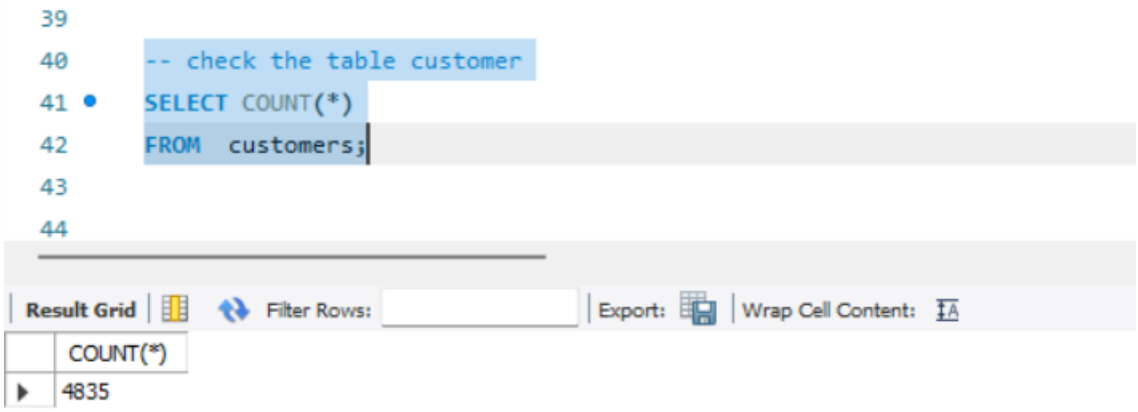
```
25
26  -- Don't make `Customer ID` as a PRIMARY KEY because in superstore_raw
27  -- there are dulicate `Customer ID`
28  INSERT INTO customers
29  SELECT DISTINCT
30      `Customer ID`,
31      `Customer Name`,
32      Segment,
33      Country,
34      City,
35      State,
36      `Postal Code`,
37      Region
38  FROM superstore_raw;
39
```

Context Help | Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
18	11:51:23	CREATE TABLE customers (customer_id VARCHAR(20) PRIMARY KEY, customer_name VARCHAR(1...	0 row(s) affected	0.078 sec
19	12:01:52	INSERT INTO customers SELECT DISTINCT 'Customer ID', 'Customer Name', Segment, Country, ...	Error Code: 1062. Duplicate entry 'TB-21520' for key 'customers.PRIMARY'	0.047 sec
20	12:05:50	SELECT COUNT(*) FROM customers LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
21	12:11:34	INSERT INTO customers SELECT DISTINCT 'Customer ID', 'Customer Name', Segment, Country, ...	Error Code: 1062. Duplicate entry 'TB-21520' for key 'customers.PRIMARY'	0.031 sec
22	12:30:36	TRUNCATE TABLE customers	0 row(s) affected	0.031 sec
23	12:36:10	DROP TABLE customers	0 row(s) affected	0.062 sec
24	12:37:30	CREATE TABLE customers (customer_id VARCHAR(20), customer_name VARCHAR(100), segment ...	0 row(s) affected	0.031 sec
25	12:43:06	INSERT INTO customers SELECT DISTINCT 'Customer ID', 'Customer Name', Segment, Country, ...	4835 row(s) affected Records: 4835 Duplicates: 0 Warnings: 0	0.125 sec



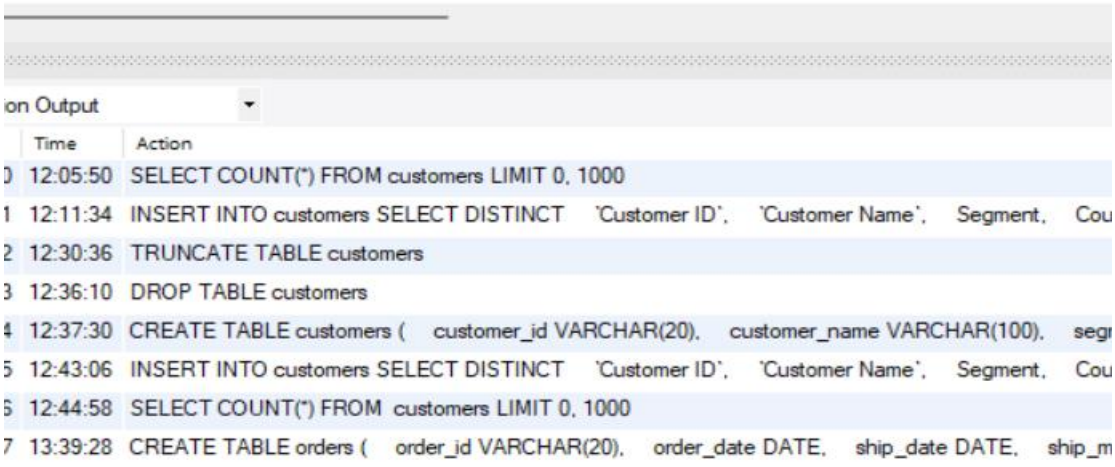
```
39
40      -- check the table customer
41  •   SELECT COUNT(*)
42      FROM  customers;
43
44
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	COUNT(*)
▶	4835

b. orders

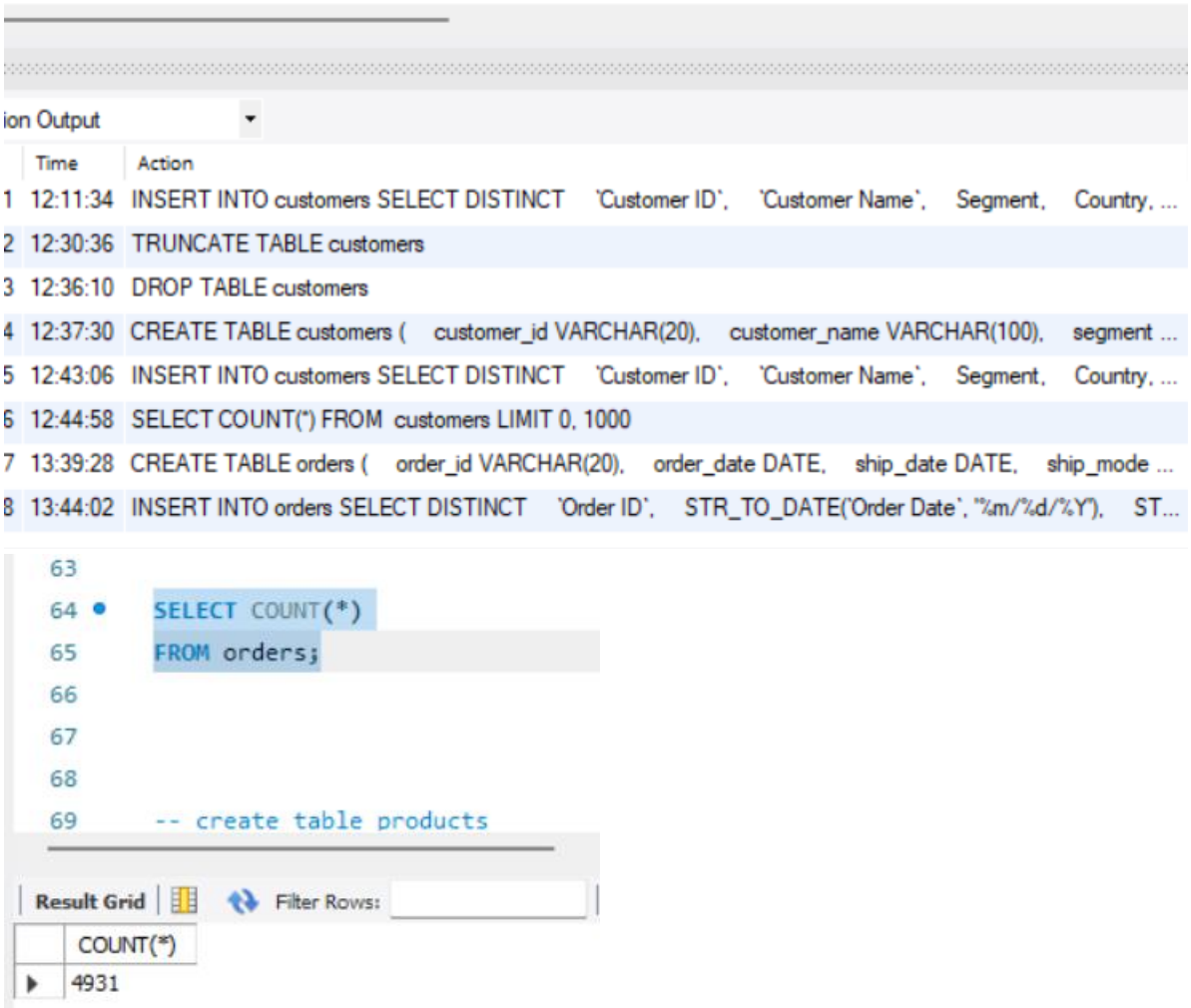
```sql
-- create table orders
CREATE TABLE orders (
    order_id VARCHAR(20),
    order_date DATE,
    ship_date DATE,
    ship_mode VARCHAR(50),
    customer_id VARCHAR(20)
);
```



ion Output

	Time	Action
0	12:05:50	SELECT COUNT(*) FROM customers LIMIT 0, 1000
1	12:11:34	INSERT INTO customers SELECT DISTINCT `Customer ID`, `Customer Name`, Segment, Cou
2	12:30:36	TRUNCATE TABLE customers
3	12:36:10	DROP TABLE customers
4	12:37:30	CREATE TABLE customers (customer_id VARCHAR(20), customer_name VARCHAR(100), segr
5	12:43:06	INSERT INTO customers SELECT DISTINCT `Customer ID`, `Customer Name`, Segment, Cou
6	12:44:58	SELECT COUNT(*) FROM customers LIMIT 0, 1000
7	13:39:28	CREATE TABLE orders (order_id VARCHAR(20), order_date DATE, ship_date DATE, ship_m

```sql
-- Insert values in orders table
INSERT INTO orders
SELECT DISTINCT
    `Order ID`,
    STR_TO_DATE(`Order Date`, '%m/%d/%Y'),
    STR_TO_DATE(`Ship Date`, '%m/%d/%Y'),
    `Ship Mode`,
    `Customer ID`
FROM superstore_raw;
```



c. products

```sql

-- create table products
CREATE TABLE products (
    order_id VARCHAR(20),
    customer_id VARCHAR(20),
    customer_name VARCHAR(100),
    product_id VARCHAR(30),
    category VARCHAR(50),
    sub_category VARCHAR(50),
    product_name VARCHAR(200),
    sales DOUBLE,
    quantity INT,
    discount DOUBLE,
    profit DOUBLE
);

-- Insert values in table products
```

Output

Action Output

#	Time	Action
29	13:48:01	CREATE TABLE products (order_id VARCHAR(20), product_id VARCHAR(30), categ
30	13:52:34	INSERT INTO products SELECT DISTINCT `Order ID`, `Product ID`, Category, `Sul
31	13:54:52	SELECT COUNT(*) FROM products LIMIT 0, 1000
32	13:55:35	CREATE DATABASE celebal_week3
33	13:55:43	SELECT COUNT(*) FROM orders LIMIT 0, 1000
34	13:59:39	SELECT * FROM products WHERE sales > (SELECT AVG(sales) FROM products) LIMIT 0, 1
35	14:33:43	DROP TABLE products
36	14:35:16	CREATE TABLE products (order_id VARCHAR(20), customer_id VARCHAR(20), cust

```sql
-- Insert values in table products
INSERT INTO products
SELECT DISTINCT
    `Order ID`,
    `Customer ID`,
    `Customer Name`,
    `Product ID`,
    Category,
    `Sub-Category`,
    `Product Name`,
    Sales,
    Quantity,
    Discount,
    Profit
FROM superstore_raw;
```

Output

Action Output

#	Time	Action
30	13:52:34	INSERT INTO products SELECT DISTINCT `Order ID`, `Product ID`, Category,
31	13:54:52	SELECT COUNT(*) FROM products LIMIT 0, 1000
32	13:55:35	CREATE DATABASE celebal_week3
33	13:55:43	SELECT COUNT(*) FROM orders LIMIT 0, 1000
34	13:59:39	SELECT * FROM products WHERE sales > (SELECT AVG(sales) FROM products) LIMIT
35	14:33:43	DROP TABLE products
36	14:35:16	CREATE TABLE products (order_id VARCHAR(20), customer_id VARCHAR(20),
37	14:37:49	INSERT INTO products SELECT DISTINCT `Order ID`, `Customer ID`, `Customer

```sql
-- check products
SELECT COUNT(*)
FROM products;
```

Result Grid | Filter Rows: | Expor

COUNT(*)
9693

3. Insert data into these tables using **SELECT DISTINCT**.

Step 2: Perform Required Queries

Write and execute SQL queries for each of the following:

1. Find all orders where sales are greater than the average sales. *(Subquery)*

```sql
-- 1.  Find all orders where sales are greater than the average sales. (Subquery)
SELECT *
FROM products
WHERE sales > (SELECT AVG(sales) FROM products);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

order_id	product_id	category	sub_category	product_name	sales
CA-2016-152156	FUR-BO-10001798	Furniture	Bookcases	Bush Somerset Collection Bookcase	261.96
CA-2016-152156	FUR-CH-10000454	Furniture	Chairs	Hon Deluxe Fabric Upholstered Stacking Chairs,...	731.94
US-2015-108966	FUR-TA-10000577	Furniture	Tables	Bretford CR4500 Series Slim Rectangular Table	957.5775
CA-2014-115812	TEC-PH-10002275	Technology	Phones	Mitel 5320 IP Phone VoIP phone	907.152
CA-2014-115812	FUR-TA-10001539	Furniture	Tables	Chromcraft Rectangular Conference Tables	1706.184
CA-2014-115812	TEC-PH-10002033	Technology	Phones	Konftel 250 Conference phone - Charcoal black	911.424

2. Find the highest sales order for each customer. *(Subquery)*

```sql
-- 2.   Find the highest sales order for each customer. (Subquery)
SELECT p.customer_id, p.order_id, p.sales
FROM products p
JOIN (
    SELECT customer_id, MAX(sales) AS max_sales
    FROM products
    GROUP BY customer_id
) t
ON p.customer_id = t.customer_id
AND p.sales = t.max_sales;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	order_id	sales
CG-12520	CA-2016-152156	731.94
SO-20335	US-2015-108966	957.5775
BH-11710	CA-2014-115812	1706.184
EB-13870	CA-2015-106320	1044.63
TB-21520	US-2015-150630	3083.43
GH-14485	CA-2016-117590	1097.544
SN-20710	CA-2015-117415	532.3992
JM-15265	CA-2016-105816	1029.95
TB-21055	CA-2016-111682	319.41
BS-11590	CA-2014-106376	1113.024
PA-19060	CA-2015-129476	339.96
HA-14920	CA-2015-110744	671.93

3. Calculate total sales for each customer. *(CTE)*

```sql
-- 3.   Calculate total sales for each customer. (CTE)
WITH total_sales_of_each_customer AS (
    SELECT customer_id, customer_name, ROUND(SUM(sales), 3) AS total_sales
    FROM products
    GROUP BY customer_id, customer_name
)
SELECT *
FROM total_sales_of_each_customer;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	customer_name	total_sales
CG-12520	Claire Gute	1148.78
DV-13045	Darrin Van Huff	1119.483
SO-20335	Sean O'Donnell	2602.576
BH-11710	Brosina Hoffman	6255.351
AA-10480	Andrew Allen	1782.872
IM-15070	Irene Maddox	4930.474
HP-14815	Harold Pawlan	1990.314
PK-19075	Pete Kriz	8158.654
AG-10270	Alejandro Grove	2565.258
ZD-21925	Zuschuss Donatelli	1493.944
KB-16585	Ken Black	2742.758
SF-20065	Sandra Flanagan	763.546
FB-13870	Emily Burns	2767.219

Result 16

4. Find customers whose total sales are above average. *(CTE + Subquery)*

```sql
-- 4.   Find customers whose total sales are above average. (CTE + Subquery)
WITH total_sales_of_customers AS (
    SELECT customer_id, customer_name, ROUND(SUM(sales), 3) AS total_sales
    FROM products
    GROUP BY customer_id, customer_name
)
SELECT *
FROM total_sales_of_customers
WHERE total_sales > (SELECT AVG(total_sales) FROM total_sales_of_customers);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	customer_name	total_sales
BH-11710	Brosina Hoffman	6255.351
IM-15070	Irene Maddox	4930.474
PK-19075	Pete Kriz	8158.654
TB-21520	Tracy Blumstein	4730.628
MA-17560	Matt Abelman	4280.621
SN-20710	Steve Nguyen	3127.959
LC-16930	Linda Cazamias	4481.162
RA-19885	Ruben Ausman	3832.314
ES-14080	Erin Smith	4648.516
KM-16720	Kunst Miller	4909.472
KD-16270	Karen Daniels	8282.358
JE-15745	Joel Eaton	6400.455
SC-20770	Stewart Carmic...	4492.657

5. Rank all customers based on total sales. *(Window Function)*

```sql
-- 5.   Rank all customers based on total sales. (Window Function)
WITH total_sales_of_customers AS (
    SELECT customer_id, customer_name, ROUND(SUM(sales), 3) AS total_sales
    FROM products
    GROUP BY customer_id, customer_name
)
SELECT customer_id, customer_name, total_sales,
RANK() OVER (ORDER BY total_sales DESC) AS customer_rank
FROM total_sales_of_customers;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	customer_name	total_sales	customer_rank
SM-20320	Sean Miller	25043.05	1
TC-20980	Tamara Chand	19017.848	2
RB-19360	Raymond Buch	15117.339	3
TA-21385	Tom Ashbrook	14595.62	4
AB-10105	Adrian Barton	14355.611	5
SC-20095	Sanjit Chand	14142.334	6
KL-16645	Ken Lonsdale	14071.917	7
HL-15040	Hunter Lopez	12873.298	8
SE-20110	Sanjit Engle	12209.438	9
CC-12370	Christopher Conant	12129.072	10
TS-21370	Todd Sumrall	11885.871	11
GT-14710	Greg Tran	11820.12	12
BM-11140	Becky Martin	11609.9	13

6. Assign row numbers to each order within a customer. *(Window Function + PARTITION BY)*

```sql
-- 6.   Assign row numbers to each order within a customer. (Window Function + PARTITION BY)
SELECT customer_id, customer_name, order_id,
ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_id) AS row_num
FROM products;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	customer_name	order_id	row_num
AA-10315	Alex Avila	CA-2014-128055	1
AA-10315	Alex Avila	CA-2014-128055	2
AA-10315	Alex Avila	CA-2014-138100	3
AA-10315	Alex Avila	CA-2014-138100	4
AA-10315	Alex Avila	CA-2015-121391	5
AA-10315	Alex Avila	CA-2016-103982	6
AA-10315	Alex Avila	CA-2016-103982	7
AA-10315	Alex Avila	CA-2016-103982	8
AA-10315	Alex Avila	CA-2016-103982	9
AA-10315	Alex Avila	CA-2017-147039	10
AA-10315	Alex Avila	CA-2017-147039	11
AA-10375	Allen Armold	CA-2014-130729	1
AA-10375	Allen Armold	CA-2014-158064	2

Result Grid | Form Editor | Field Types

7. Display top 3 customers based on total sales. *(Window Function)*

```sql
-- Q7. Display top 3 customers based on total sales. (Window Function)
WITH customer_sales AS (
    SELECT customer_id, customer_name, SUM(sales) AS total_sales
    FROM products
    GROUP BY customer_id, customer_name
)
SELECT *
FROM (
    SELECT customer_id, customer_name, total_sales,
        RANK() OVER (ORDER BY total_sales DESC) AS customer_rank
    FROM customer_sales
) ranked_customers
WHERE customer_rank <= 3;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	customer_name	total_sales	customer_rank
SM-20320	Sean Miller	25043.05	1
TC-20980	Tamara Chand	19017.847999999998	2
RB-19360	Raymond Buch	15117.339	3

Step 3: Final Combined Query

Write **one final query** that shows:

- Customer Name
- Total Sales
- Rank

(Use **JOIN + CTE + Window Function together**)

```
168
169     -- Final Query Customer Name, Total Sales, Rank (JOIN + CTE + Window Function)
170 •   WITH customer_sales AS (
171         SELECT p.customer_id, SUM(p.sales) AS total_sales
172         FROM products p
173         GROUP BY p.customer_id
174     )
175     SELECT
176         c.customer_name, cs.total_sales,
177         RANK() OVER (ORDER BY cs.total_sales DESC) AS customer_rank
178     FROM customer_sales cs
179     JOIN customers c
180     ON cs.customer_id = c.customer_id;
181
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

customer_name	total_sales	customer_rank
Sean Miller	25043.05	1
Sean Miller	25043.05	1
Sean Miller	25043.05	1
Sean Miller	25043.05	1
Sean Miller	25043.05	1
Tamara Chand	19017.847999999998	6
Tamara Chand	19017.847999999998	6
Tamara Chand	19017.847999999998	6
Tamara Chand	19017.847999999998	6
Tamara Chand	19017.847999999998	6
Raymond Buch	15117.339	11
Raymond Buch	15117.339	11
Raymond Buch	15117.339	11

Result 23

Mini Project: Customer Sales Insights

Answer the following using SQL:

1. Who are the top 5 customers?

```
184
185     -- 1.   Who are the top 5 customers?
186     WITH total_customers_sales AS (
187         SELECT customer_id, customer_name, SUM(sales) AS total_sales
188         FROM products
189         GROUP BY customer_id, customer_name
190     )
191     SELECT *
192     FROM total_customers_sales
193     ORDER BY total_sales DESC
194     LIMIT 5;
195
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	customer_name	total_sales
SM-20320	Sean Miller	25043.05
TC-20980	Tamara Chand	19017.847999999998
RB-19360	Raymond Buch	15117.339
TA-21385	Tom Ashbrook	14595.62
AB-10105	Adrian Barton	14355.610999999997

2. Who are the bottom 5 customers?

```sql
-- 2.   Who are the bottom 5 customers?
WITH customer_sales AS (
    SELECT customer_id, customer_name, SUM(sales) AS total_sales
    FROM products
    GROUP BY customer_id, customer_name
)
SELECT *
FROM customer_sales
ORDER BY total_sales ASC
LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	customer_name	total_sales
TS-21085	Thais Sissman	4.833
LD-16855	Lela Donovan	5.304
MG-18205	Mitch Gastineau	12.32
CJ-11875	Carl Jackson	16.52
RS-19870	Roy Skaria	22.328

3. Which customers made only one order?

```
206
207     -- 3.   Which customers made only one order?
208     SELECT customer_id, customer_name, COUNT(DISTINCT order_id) AS total_orders
209     FROM products
210     GROUP BY customer_id, customer_name
211     HAVING COUNT(DISTINCT order_id) = 1;
212
213
214
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	customer_name	total_orders
AO-10810	Anthony O'Donnell	1
AR-10570	Anemone Ratner	1
CJ-11875	Carl Jackson	1
JC-15385	Jenna Caffey	1
JR-15700	Jocasta Rupert	1
LD-16855	Lela Donovan	1
MG-18205	Mitch Gastineau	1
PH-18790	Patricia Hirasaki	1
RE-19405	Ricardo Emerson	1
RM-19750	Roland Murray	1
SM-20905	Susan MacKendrick	1
TC-21145	Theresa Coyne	1

4. Which customers have above-average sales?

```sql
212
213     -- 4.   Which customers have above-average sales?
214     WITH customer_sales AS (
215         SELECT customer_id, customer_name, SUM(sales) AS total_sales
216         FROM products
217         GROUP BY customer_id, customer_name
218     )
219     SELECT *
220     FROM customer_sales
221     WHERE total_sales > (SELECT AVG(total_sales) FROM customer_sales);
222
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	customer_name	total_sales
BH-11710	Brosina Hoffman	6255.351000000001
IM-15070	Irene Maddox	4930.473999999999
PK-19075	Pete Kriz	8158.654
TB-21520	Tracy Blumstein	4730.628
MA-17560	Matt Abelman	4280.620999999999
SN-20710	Steve Nguyen	3127.9592000000007
LC-16930	Linda Cazamias	4481.162
RA-19885	Ruben Ausman	3832.3139999999994
ES-14080	Erin Smith	4648.516
KM-16720	Kunst Miller	4909.472000000001
KD-16270	Karen Daniels	8282.358
JE-15745	Joel Eaton	6400.454999999999
SC-20770	Stewart Carmic...	4492.656999999999

5. What is the highest order value per customer?

```
222
223     -- 5.   What is the highest order value per customer?
224 •   SELECT customer_id, customer_name, MAX(sales) AS highest_order_value
225     FROM products
226     GROUP BY customer_id, customer_name;
227
228
229
230
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	customer_name	highest_order_value
CG-12520	Claire Gute	731.94
DV-13045	Darrin Van Huff	721.875
SO-20335	Sean O'Donnell	957.5775
BH-11710	Brosina Hoffman	1706.184
AA-10480	Andrew Allen	479.97
IM-15070	Irene Maddox	1819.86
HP-14815	Harold Pawlan	1242.9
PK-19075	Pete Kriz	4305.552
AG-10270	Alejandro Grove	1336.44
ZD-21925	Zuschuss Donatelli	823.96
KB-16585	Ken Black	1013.832
SF-20065	Sandra Flanagan	254.9
EB-13870	Emily Burns	1044.63

Result Grid
Form Editor
Field Types